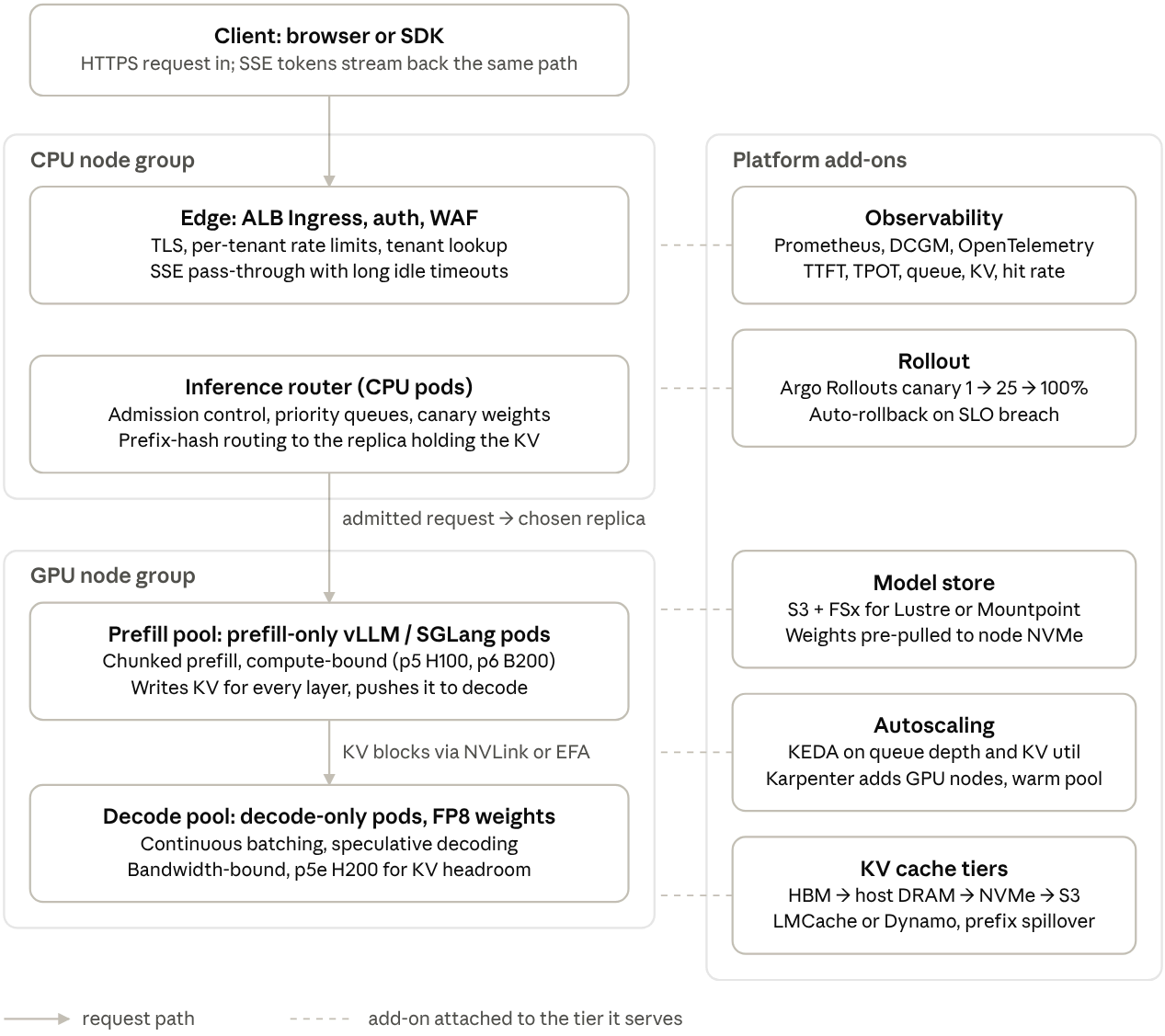
AI Inference on EKS: Architecture + Cheat Sheet

Oct 24, 2026

Reference architecture on EKS

One EKS cluster with a CPU control tier and a GPU data tier exercises all 25 problems; a request flows edge → router → prefill pool → decode pool → SSE stream back. Build it in the order of the table (edge first, kernels last) and each layer becomes a study module.

A request flows edge → router → prefill pool → decode pool, then streams back



LLM serving stack on EKS · request path and platform add-ons

The CPU node group handles auth, admission and routing; the GPU node group splits prefill from decode and moves KV between them; the add-ons on the right attach to whichever tier they serve.

Layer	On EKS	What it does	Problems
Edge	ALB Ingress or Gateway API, auth, WAF	TLS, per-tenant rate limits, SSE pass-through with long idle timeouts	2, 21

Layer	On EKS	What it does	Problems
Router	CPU Deployment: llm-d / NVIDIA Dynamo router, or your own Go service	Admission control, priority queues, prefix-hash routing to the replica that holds the KV, canary weights	8, 19, 21
Prefill pool	vLLM or SGLang pods on p5 / p5e nodes (H100 / H200), prefill-only	Chunked prefill, long-prompt isolation	7, 22
Decode pool	vLLM or SGLang pods, decode-only, quantized weights (FP8)	Continuous batching, speculative decoding, ITL tuning	6, 9, 10, 11
KV fabric	EFA networking + NIXL or LMCache; tiers HBM → host DRAM → NVMe → S3	KV transfer prefill → decode, prefix-cache spillover	8, 22
Model store	S3 + Mountpoint or FSx for Lustre; weights pre-pulled to node NVMe	Fast cold starts, pinned model versions	19, 20
Autoscaling	KEDA on queue depth and KV utilization; Karpenter for GPU nodes; warm pool	Scale under slow cold starts	20
Observability	Prometheus scraping engine /metrics, DCGM exporter, OpenTelemetry, Grafana	TTFT, TPOT, queue depth, KV util, hit rate, cost per request	13, 14, 16, 17
Rollout	Argo Rollouts canary with SLO-based abort	New model or quant with automatic rollback	19
Multi-GPU	NVIDIA GPU Operator, device plugin, EFA plugin; TP / PP / EP	MoE serving, GPU choice	12, 23
Bench harness	Kubernetes Job running vllm bench or genai-perf on real traces	Engine selection, cost and capacity math	18, 24, 25
Kernel dev	Dev pod with Triton, CUDA toolkit, Nsight Systems	Fused RMSNorm, profiling	15

1–2. Request lifecycle and streaming

TTFT is dominated by queue wait plus prefill; everything after the first token is decode, one forward pass per token.

1. Request in → first token out

1. Edge: TLS, auth, rate limit, tenant lookup.
2. Router: admission check (is there KV room?), priority queue, pick a replica (prefix hash → cache hit).
3. Engine: tokenize, allocate KV blocks, enqueue in the scheduler.
4. Scheduler: next step picks waiting prefill work plus running decode work (continuous batching).
5. Prefill: one forward pass over all prompt tokens, compute-bound, writes KV for every layer.
6. Sample first token (temperature, top-p, logit bias, grammar).
7. Detokenize and flush the first SSE chunk.
8. Decode loop: one token per step, memory-bandwidth-bound, until EOS, max_tokens, or stop string.

Interview line: "Prefill is a matmul over N tokens; decode is a matvec per token that re-reads all weights and the KV cache. That is why they need different optimizations."

2. SSE streaming, disconnects, cancellation

- Response headers: `Content-Type: text/event-stream`, `Cache-Control: no-cache`, `X-Accel-Buffering: no`; disable proxy buffering at every hop (ALB idle timeout ≥ max generation time).
- Each chunk: `data: {json}\n\n`; send a heartbeat comment line every 10–15 s so idle proxies do not cut the stream.
- Detect disconnect: the write to the socket fails or the request context is cancelled; in Python, `await request.is_disconnected()` inside the generator loop.
- Cancel in flight: call the engine's abort (`engine.abort(request_id)`) so the scheduler frees KV blocks on the next step; never let a dead client hold GPU memory.
- Make the abort idempotent and bounded (a request that is already finished is a no-op).
- Client resume: send `id:` on each event and accept `Last-Event-ID`, or make the client re-send the prompt and rely on prefix caching for a cheap restart.

3–5. KV cache, PagedAttention, FlashAttention

A Llama-3-70B KV cache costs 320 KB per token in FP16, so 100 users at 32K context need about 1.07 TB before weights; GQA already saved you 8× versus MHA.

3. Size the KV cache

```
\text{KV bytes/token} = 2 \times L \times H_{kv} \times d_{head} \times \text{bytes}
```

Term	Llama-3-70B	Notes
Layers L	80	
KV heads H_kv	8	64 query heads → GQA group of 8
Head dim d_head	128	hidden 8192 / 64 heads
Bytes	2 (FP16/BF16)	1 for FP8 KV
Per token	2 × 80 × 8 × 128 × 2 = 327,680 B ≈ 320 KB	MHA would be 2.5 MB
Per user at 32K	32,768 × 320 KB ≈ 10 GB	
100 users	≈ 1.07 TB FP16, ≈ 0.54 TB FP8	
Weights	70B × 2 B = 140 GB (FP16), 70 GB (FP8)	

Worked answer: on 8× H100 (640 GB) with FP8 weights (70 GB) you have ~550 GB free; FP8 KV fits 100 users near full context, FP16 KV fits about 50. Then say the real sizing uses the *average* context (often 2–8K), not the max, with admission control for the tail.

4. PagedAttention

- Problem: engines pre-allocated a contiguous KV buffer of max_len per request; 60–80% sat unused (internal fragmentation) and you could not fit a new request between two existing ones (external fragmentation).
- Fix: split KV into fixed blocks (16 tokens in vLLM), keep a per-request block table like an OS page table, allocate on demand, free on finish.

- Payoff: near-zero waste, 2–4× more concurrent requests, copy-on-write sharing for beam search and parallel sampling, and the foundation for prefix caching (a shared block is just a shared page).
- Cost: the attention kernel gathers blocks via the table, a small indirection overhead.

5. FlashAttention

- Same FLOPs, different memory traffic: naive attention writes the N×N score matrix to HBM, softmaxes it, reads it back, then multiplies by V. For N = 8K that matrix is 128 MB per head per layer.
- FlashAttention tiles Q, K, V into SRAM-sized blocks, computes softmax incrementally with the online (running max and sum) trick, and never materializes the score matrix.
- Attention is memory-bound, so cutting HBM reads/writes from $O(N^2)$ to $O(N)$ gives 2–4× speedup and $O(N)$ memory.
- Backward pass recomputes scores from saved logsumexp instead of storing them, which is why training memory drops too.
- FA3 on Hopper adds warp specialization, TMA, and FP8; the same idea, more overlap.

6–8. Scheduling: batching, chunked prefill, prefix caching

The scheduler decides every step what goes on the GPU; the three wins are never idling on finished sequences, never letting one long prompt block the step, and never recomputing a prefix you already have.

6. Continuous batching under latency deadlines

- Static batching: wait for B requests, run until the longest finishes; GPU idles on padding and finished rows, TTFT includes the wait for the batch to fill.
- Continuous (iteration-level) batching: re-plan the batch every decode step; a finished sequence leaves, a waiting one joins, no padding. Throughput 2–10× over static at the same latency.
- Deadline-aware design: give each request a TTFT and TPOT budget; the scheduler admits new prefill only while the running batch's projected step time keeps TPOT under budget (`max_num_batched_tokens` is the knob). Use EDF or priority ordering in the waiting queue; preempt (swap or recompute) the lowest priority when KV runs out.
- Watch: larger batches raise TPOT because each step does more work; the knee is where step time stops being bandwidth-bound and starts being compute-bound.

7. Long prompts stall short chats: chunked prefill

- Root cause: a 100K-token prefill is one huge compute-bound step; every decode request in the batch waits that whole step, so their ITL spikes and new requests' TTFT climbs.
- Fix: split prefill into chunks (2–8K tokens) and co-schedule one chunk with the running decode tokens each step. Each step is bounded, decode keeps flowing, and the long prompt's own TTFT grows only slightly.
- Policy: decode-first (fill the token budget with decode, then as much prefill as fits), cap prefill share per step, and route prompts above a threshold to a dedicated prefill pool (see 22).
- Also: set `max_model_len` per tenant and reject or queue prompts that would starve the step.

8. Prefix caching and cache-aware routing

- Engine level: hash each full KV block by its token content plus the hash of the previous block; keep an LRU of blocks; a new request walks its prompt block by block and reuses hits. System prompts, few-shot examples, and multi-turn history become free prefill.
- Routing level: replicas do not share HBM, so the router must send a request to the replica that already holds its prefix. Options: (a) consistent hashing on the first K tokens or a session id, (b) a global prefix index (radix tree of prompt hashes → replica, as in SGLang's cache-aware router or llm-d's EPP) and score replicas by estimated hit length minus load, (c) sticky sessions as the cheap fallback.
- Balance hit rate against hotspots: score = α × prefix_match − β × queue_depth − γ × KV_util; fall back to the least-loaded replica when the match is short.
- Spill to tiers: evicted GPU blocks go to host memory then NVMe (LMCache, Dynamo KV manager); a cross-replica hit from DRAM beats recompute for prompts over a few thousand tokens.
- Measure: prefix-cache hit rate per tenant, prefill tokens avoided, TTFT for cached vs uncached.

9–11. Latency levers: ITL, speculative decoding, quantization

Decode is bandwidth-bound, so inter-token latency falls when each step moves fewer bytes (quantize, cache better) or produces more tokens per byte moved (speculate, batch).

9. Cut inter-token latency without tanking throughput

- Decode step time ≈ (weights + KV bytes read) / HBM bandwidth + kernel launch overhead. A 70B FP16 model on one H100-class node reads 140 GB per step; at ~3.3 TB/s that is ~40 ms floor before batching.
- Levers, in order of cheapness: FP8 weights and KV (halves bytes), CUDA graphs to kill launch overhead at small batch, tensor parallel across more GPUs (bandwidth adds up, interconnect cost rises), speculative decoding, cap batch size per replica and add replicas.
- Throughput-preserving trick: keep batches large but bound step time with `max_num_batched_tokens`; separate latency-sensitive tenants onto their own pool rather than shrinking batches everywhere.

10. Speculative decoding: when it helps and hurts

- Idea: a cheap drafter proposes k tokens; the target model verifies all k in one forward pass; accept the longest matching prefix plus one corrected token. Output distribution is unchanged (rejection sampling).
- Helps when: batch is small (GPU has spare compute, decode is bandwidth-bound), acceptance rate is high (code, structured text, repetitive domains), drafter is ⩾10× cheaper (n-gram lookup, EAGLE/Medusa heads, a 1B draft model).
- Hurts when: batch is already large (verification steals compute from other users, throughput drops), acceptance is low (creative text, high temperature), or the drafter adds its own latency comparable to the saved steps.
- Rule of thumb: speedup ≈ (expected accepted + 1) / (1 + drafter cost ratio); measure acceptance rate per workload and turn it off above a batch-size threshold.

11. Quantization: FP8 vs INT8 vs INT4 vs NVFP4

Format	Bits	Hardware	Quality	Use
FP8 (E4M3)	8	Hopper, Blackwell tensor cores	≈ lossless with per-tensor or per-channel scales	Default for weights, activations, KV
INT8 (W8A8, SmoothQuant)	8	Ampere and up	Small loss; outlier channels need smoothing	Older GPUs

Format	Bits	Hardware	Quality	Use
INT4 (AWQ, GPTQ, W4A16)	4	Any	0.5–2 pt loss on hard tasks, worse on small models	Memory-bound, small batch, 1–2 GPU serving
NVFP4	4	Blackwell	Near-FP8 quality via two-level (block 16, FP8 block scale) scaling	Blackwell, large batch

- Pick: on H100/H200 at scale, FP8 W8A8 with FP8 KV; on Blackwell, NVFP4 for weights when memory-bound; INT4 only when you must fit a model on fewer GPUs.
- Measure loss, not vibes: perplexity on a held-out set, task accuracy on your eval set (MMLU-style plus your own prompts), KL divergence between FP16 and quantized logits, and a human or LLM-judge pairwise win rate on 200+ real prompts. Ship when the delta is inside your noise band.

12–15. Hardware, profiling, kernels

Pick the GPU by the bottleneck: decode wants HBM bandwidth and capacity, prefill wants FLOPs, and cheap small-model serving wants cost per token, not peak specs.

12. Pick GPUs for a workload

GPU	Memory	Bandwidth	Best for	EKS instance
H100	80 GB HBM3	~3.3 TB/s	Balanced, widest software support	p5
H200	141 GB HBM3e	~4.8 TB/s	Long context, bigger KV, fewer TP shards	p5e / p5en
B200	~180 GB HBM3e	~8 TB/s	Max throughput, NVFP4, MoE, FP4 kernels	p6-b200
L40S	48 GB GDDR6	~0.86 TB/s	7–13B models, embeddings, cost-sensitive	g6e

- Decode-heavy chat at 70B: H200 (KV fits, fewer shards). Prefill-heavy RAG or summarization: B200 or H100 by FLOPs per dollar. Small models under 13B: L40S at a fraction of the price.
- Rule: compute the bytes per step and tokens per second you need, then price per million tokens per GPU, not price per hour.

13. Profile with Nsight: which of the three?

- Nsight Systems (nsys) first: the timeline shows gaps between kernels (launch overhead or CPU-bound scheduler), long kernels (compute or memory), and copies (H2D/D2H).
- Nsight Compute (ncu) on the hot kernel: look at SM throughput vs DRAM throughput. High DRAM %, low SM % → memory-bound (typical decode). High SM %, low DRAM → compute-bound (prefill). Both low with many tiny kernels → launch overhead.
- Roofline check: arithmetic intensity = FLOPs / bytes; below the ridge point (~300 FLOP/B on H100 FP16) you are bandwidth-bound.
- Fixes map 1:1: memory-bound → quantize, fuse, reduce KV reads; compute-bound → FP8 tensor cores, bigger tiles, chunk prefill; launch-bound → CUDA graphs, kernel fusion, bigger batches.

14. GPU at 30% utilization under load

1. Confirm the metric: nvidia-smi "utilization" only says a kernel was running, not how busy the SMs were; use DCGM SM activity and tensor-core activity.
2. Check the CPU: tokenizer, Python scheduler, sampling, detokenization, or the HTTP server saturated? One process per GPU, pin CPUs, use the engine's async or multiprocess front end.
3. Check batch size: if requests arrive serially and batch stays at 1–2, the GPU waits on bandwidth; raise concurrency or add prefix caching.
4. Check kernel launches: small batches without CUDA graphs spend more time launching than computing.
5. Check memory: KV cache full means the scheduler is preempting or waiting; raise `gpu_memory_utilization`, quantize KV, or add replicas.
6. Check I/O: data loading, logging, metrics scrape on the hot path, or a sync `.item()` call stalling the stream.
7. Check topology: TP across PCIe instead of NVLink, or EFA not enabled for multi-node.

15. Fused RMSNorm kernel

RMSNorm(x) = x / sqrt(mean(x²) + ε) × w. PyTorch's eager version launches several kernels (square, mean, rsqrt, mul, mul) and round-trips the row through HBM each time; a fused kernel reads the row once and writes once.

```
@triton.jit
def rmsnorm_kernel(X, W, Y, stride, N, eps, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK)
    mask = cols < N
    x = tl.load(X + row * stride + cols, mask=mask, other=0.0).to(tl.float32)
    var = tl.sum(x * x, axis=0) / N
    rstd = 1.0 / tl.sqrt(var + eps)
    w = tl.load(W + cols, mask=mask, other=0.0).to(tl.float32)
    y = x * rstd * w
    tl.store(Y + row * stride + cols, y.to(Y.dtype.element_ty), mask=mask)
```

- Launch grid = (rows,), BLOCK = next power of two ⩾ hidden size; accumulate in FP32, store in BF16.
- Benchmark with `triton.testing.do_bench` against `torch.nn.functional.rms_norm` and a compiled version; report GB/s achieved vs HBM peak, not just ms. Expect 2–4× over eager and near-bandwidth-bound at 8K hidden.
- Mention: fuse the residual add in too (`x = x + residual` before the norm) for one more HBM pass saved per layer.

16–17. Observability and tail latency

Track the four engine signals (TTFT, TPOT, queue depth, KV utilization) per replica and per tenant, and the two business ones (prefix-cache hit rate, cost per request); a 10× p99/p50 gap almost always comes from queueing or one slow path, not from the GPU being slow.

16. Observability design

Metric	Source	Alert on
TTFT p50/p95/p99	Gateway span, engine histogram	p95 > SLO for 5 min
TPOT / ITL p50/p99	Per-token timestamps in the stream	p99 > 2× p50
Queue depth, wait time	Engine `num_requests_waiting`	Depth rising with flat throughput

Metric	Source	Alert on
KV cache utilization	Engine `gpu_cache_usage_perc`	> 90% sustained, preemptions > 0
Prefix-cache hit rate	Engine prefix hit counters, per tenant	Drop after a deploy or router change
Batch size, tokens per step	Engine scheduler stats	Batch stuck at 1–2 under load
GPU SM / tensor / DRAM activity	DCGM exporter	SM activity < 40% with queue > 0
Tokens in / out per request	Gateway	Cost model inputs
Cost per request	(GPU $/h ÷ tokens/s) × tokens, per tenant	Weekly trend
Errors, aborts, timeouts	Gateway, engine	Abort rate > 1%

- Stack on EKS: engine `/metrics` → Prometheus (or Amazon Managed Prometheus) → Grafana; OpenTelemetry traces with spans for queue, prefill, each decode chunk; logs with request_id, tenant, model version, replica.
- Tag everything with model version and quantization so a canary can be compared in one query.
- Derive SLOs per tenant tier: interactive chat TTFT p95 < 500 ms and ITL p95 < 50 ms; batch jobs on throughput only.

17. p99 is 10× p50: diagnose it

1. Split by phase: is the tail in queue wait, prefill, or decode? Traces answer this in one query.
2. Queue wait tail → bursty arrivals or head-of-line blocking; add replicas, tighten admission, use priority queues, chunk long prefills.
3. Prefill tail → long prompts in the same batch as short ones (problem 7) or prefix-cache misses for one tenant.
4. Decode tail → preemption and recompute when KV runs out (`num_preemptions` rising), swap to CPU, or a replica stuck at a huge batch.
5. Replica tail → one bad node: thermal throttling, a failing GPU, ECC retries, a noisy TP neighbour; compare per-replica p99 and drain the outlier.

6. Client tail → slow consumers back-pressuring the stream, or long max_tokens generations counted end to end; report TTFT and ITL, not just total latency.
7. Infra tail → garbage collection or tokenizer stalls in the Python front end, DNS or load-balancer timeouts, pod CPU throttling (check `container_cpu_cfs_throttled_seconds`).

Say out loud: "Tail latency is a scheduling and isolation problem; I would look at the per-phase histograms and the per-replica split before touching the model."

18–20. Engines, rollouts, autoscaling

Benchmark engines on your own traffic shape, roll models out as a weighted canary with automatic abort, and treat GPU autoscaling as a capacity-forecasting problem because a cold start takes minutes, not seconds.

18. vLLM vs TensorRT-LLM vs SGLang

	vLLM	TensorRT-LLM	SGLang
Strength	Widest model and hardware coverage, fast to adopt, big community	Peak throughput on NVIDIA, FP8/FP4 kernels, in-flight batching, strong on B200	RadixAttention prefix cache, structured output, cache-aware router, strong multi-turn and agent workloads
Cost	Slightly below TRT-LLM at peak	Engine build per model and shape, NVIDIA-only, slower iteration	Smaller ecosystem than vLLM
Pick when	Many models, mixed hardware, fast-moving team	One or two models, NVIDIA fleet, squeeze last 10–30%	Heavy shared prefixes, agents, JSON/grammar output

- Benchmark method: replay a real trace (prompt and output length distributions, arrival times, prefix sharing) with `vllm bench serve` or `genai-perf`; sweep concurrency; plot throughput vs TTFT p95 and ITL p95; compare at the latency SLO, not at max throughput.
- Also score: time to add a new model, quantization support, observability, and the failure modes you saw during the run.

19. Roll out a new model or quantization safely

1. Pre-flight offline: eval suite (accuracy, refusal rate, format compliance), quant loss vs baseline (problem 11), load test for TTFT/ITL at target concurrency.
2. Shadow: mirror 1–5% of traffic to the new version, discard responses, compare latency and a judge-scored quality sample.
3. Canary: Argo Rollouts with Gateway API traffic split 1% → 5% → 25% → 100%; each step gated on TTFT p95, ITL p95, error rate, prefix-cache hit rate, and a quality judge over sampled responses.
4. Keep both versions warm on separate pools so rollback is a weight flip, not a cold start; weights pinned by content hash in S3, never "latest".
5. Automatic abort to the previous weight on any gate failure; manual kill switch at the router.
6. Watch for subtle breaks: tokenizer changes, chat template drift, stop-token behaviour, max context changes, and client-visible differences in streaming chunking.

20. GPU autoscaling with slow cold starts

- Cold start = node provisioning (2–5 min) + image pull (minutes for 10 GB+ images) + weight load (140 GB over the network) + CUDA graph capture and warm-up. Total 5–15 min if you do nothing.
- Shrink it: pre-pull images and weights onto node NVMe (daemonset or Karpenter startup script), use FSx for Lustre or S3 Mountpoint with local cache, slim images, snapshot-restored containers, keep a warm pool of idle GPU nodes (Karpenter with over-provisioning pods).
- Scale signal: not CPU. Use KEDA on queue depth, wait time, or KV utilization from Prometheus; scale out early on the slope of arrivals, scale in slowly (10–20 min cooldown) to avoid flapping.
- Predict: schedule capacity by time of day from the last weeks' traffic; autoscaling handles the residual.
- Protect: admission control sheds or queues low-priority work while new replicas come up; never let autoscaling be the only defence against a burst.

21–23. Multi-tenant, disaggregation, MoE parallelism

Isolation is the theme: tenants get fair shares and admission limits, prefill and decode get their own GPUs so one does not jitter the other, and MoE layers get expert parallelism with the all-to-all kept inside an NVLink domain wherever possible.

21. Multi-tenant serving

- Admission control at the router: per-tenant token budgets (tokens per minute, concurrent requests, max context); reject with 429 and `Retry-After` when over; estimate a request's cost as prompt tokens + expected output tokens before admitting.
- Fair scheduling: weighted fair queuing on tokens, not requests (one 100K prompt is not one unit). Deficit round robin per tenant with a token quantum; track cumulative tokens served and prefer the tenant furthest below its share.
- Priority queues: tiers (interactive > standard > batch); strict priority with aging so batch never starves; preempt batch sequences first when KV is full.
- Isolation: separate pools for latency-critical tenants; cap any one tenant's share of a replica's KV; per-tenant prefix-cache accounting so one tenant's cache churn does not evict another's.
- Observability per tenant: queue wait, TTFT, tokens served vs quota, rejection rate.

22. Disaggregated prefill and decode

- Why: prefill is compute-bound and bursty, decode is bandwidth-bound and steady. Co-locating them makes decode ITL jitter whenever a prefill lands, and forces one batch policy on two different workloads. Separate pools let each be sized and tuned alone (prefill on B200/H100 FLOPs, decode on H200 bandwidth and capacity, different TP degrees).
- How the KV moves: after prefill, the KV blocks for the prompt are pushed to the chosen decode replica over NVLink (same node), RDMA/EFA (across nodes), or via a shared tier (host DRAM or NVMe pool). Libraries: NIXL (NVIDIA Dynamo), LMCache, vLLM KV connectors, Mooncake. Overlap the transfer layer by layer with the remaining prefill so the decode side can start as soon as the last layer lands.
- Cost: for a 10K-token prompt on Llama-70B that is ~3.2 GB of FP16 KV; at ~50 GB/s per EFA link that is tens of ms, so disaggregate for long prompts and high load, not for tiny chats. Add router complexity, two pools to autoscale, and the risk of a decode replica sitting idle while prefill catches up.
- Pay off when: prompt lengths are long or bimodal, ITL SLO is tight, and you run enough GPUs that pool-level specialisation beats a single pool's flexibility.

23. Large MoE: TP, PP, EP, and all-to-all

- Tensor parallel (TP): split each matmul across GPUs; needs all-reduce every layer, so keep it inside an NVLink domain (8 GPUs). Cuts per-GPU weight bytes and latency.

- Pipeline parallel (PP): split layers across GPUs or nodes; only activations cross the boundary, cheap on bandwidth, but adds bubbles and inter-stage latency; good across nodes.
- Expert parallel (EP): put different experts on different GPUs; the attention block runs replicated or TP, then tokens are routed to their top-k experts with an all-to-all dispatch and an all-to-all combine per MoE layer.
- Typical mix for a DeepSeek-V3-class model (671B, 256 experts, 8 active): TP within the node for attention, EP across 8–32 GPUs for experts, PP only when the model still does not fit. Prefill prefers wide EP for throughput; decode may prefer smaller groups to cut all-to-all latency.
- All-to-all overhead: every token sends its hidden state (hidden × bytes) to k experts and gets k results back, twice per MoE layer. Cost scales with tokens × k × hidden × bytes ÷ interconnect bandwidth, and with load imbalance (hot experts become stragglers). Mitigate with expert load balancing (auxiliary loss at training, expert replication of hot experts at serving), FP8 dispatch, overlapping communication with the next micro-batch's compute (DeepEP-style), and keeping EP inside NVLink or on EFA with GPUDirect RDMA.

24–25. Cost and capacity planning

Cost per million tokens = GPU dollars per hour ÷ tokens per hour actually served, so the two biggest levers are utilization and bytes per token; everything else is a 10–30% trim.

24. Cut cost per million tokens by 70% without hurting quality

Lever	Typical saving	Quality risk
Raise utilization: time-of-day scaling, fill valleys with batch jobs, bigger batches	1.5–2.5×	None
FP8 weights + FP8 KV (NVFP4 on Blackwell)	1.5–2×	Low, verify with evals
Prefix caching for system prompts and history	30–80% of prefill	None
Pricing: reserved, capacity blocks, spot for batch pools	30–60%	None

Lever	Typical saving	Quality risk
Speculative decoding at low batch	1.5–2× on ITL-bound pools	None if verified
Model routing: small model for easy requests, big for hard	20–50%	Needs a router eval
Cap max_tokens, trim prompts, cache identical responses	10–30%	Low
Disaggregate and right-size GPU per phase	10–30%	None

Stack three: utilization ×2, FP8 ×1.5, prefix caching and pricing ×1.2 → cost ÷ 3.6, a 72% cut. Guard with the same eval gate as a model rollout (problem 19) and track quality per tenant weekly.

25. ChatGPT-style inference for 1M daily active users

Assumptions (state them, then compute):

Input	Value
Messages per user per day	10
Prompt tokens per message (with history)	1,000
Output tokens per message	300
Peak-to-average ratio	3×
Model	70B-class, FP8, 8× H100 per replica
Replica capacity	~128 concurrent streams, ~3,000 output tok/s, ~20,000 prefill tok/s

Compute:

1. Requests: 1M × 10 = 10M per day → average 116 req/s → peak ~350 req/s.
2. Tokens: 3B output and 10B prompt tokens per day; with a 70% prefix-cache hit rate, ~3B prompt tokens actually prefilled.
3. Concurrency (Little's law): request duration ≈ 0.5 s TTFT + 300 × 0.04 s ≈ 12.5 s → 350 × 12.5 ≈ 4,400 concurrent streams at peak.

4. Decode replicas: 4,400 ÷ 128 ≈ 35 replicas.
5. Prefill: 350 req/s × 1,000 tokens × 30% miss ≈ 105K tok/s ÷ 20K ≈ 6 replicas (18 with no caching).
6. Headroom and N+2: ~50 replicas → 400 H100s at peak, scaled to ~15–20 overnight.
7. Cost: 50 × 8 GPUs × ~$3–4 per GPU-hour (approximate, committed pricing) ≈ $1.4M–1.9M per month at peak sizing; with diurnal scaling closer to $0.8M–1.2M.
8. Cost per million output tokens: $1.2M ÷ 90B ≈ $13 all-in; a fully utilized replica alone is ~$3–5, which is why utilization dominates (problem 24).

Finish with what you would also build: a prefill pool on cheaper FLOPs, a small-model router, prefix-aware routing, and a batch tier for the overnight valley. Flag that every number is an assumption to be replaced by your real trace.

Numbers and formulas to memorize

Four formulas and a dozen constants cover most back-of-envelope questions in this list.

```
\text{KV bytes/token} = 2 \, L \, H_{kv} \, d_{head} \, b
```

```
\text{Decode step time} \approx \frac{\text{weight bytes} + \text{KV bytes read}}{\text{HBM bandwidth}}
```

```
\text{Prefill FLOPs} \approx 2 \, P \, N_{tokens}
```

```
\text{Concurrency} = \text{arrival rate} \times \text{request duration} \quad (\text{Little's law})
```

Constant	Value
Llama-3-70B	80 layers, 64 heads, 8 KV heads, d_head 128, hidden 8192
KV per token, 70B FP16	~320 KB (FP8: 160 KB; MHA would be 2.5 MB)
Weights 70B	140 GB FP16, 70 GB FP8, ~40 GB INT4
H100 / H200 / B200 memory	80 / 141 / ~180 GB
H100 / H200 / B200 bandwidth	~3.3 / ~4.8 / ~8 TB/s

Constant	Value
L40S	48 GB, ~0.86 TB/s
H100 FP8 dense tensor	~1 PFLOPS per GPU (peak, expect 30–50% MFU)
NVLink per GPU (Hopper)	~900 GB/s total
EFA per p5 node	3.2 Tb/s aggregate (32 × 100 Gb/s)
vLLM KV block	16 tokens
Roofline ridge, H100 FP16	~300 FLOP per byte
Interactive SLO (typical)	TTFT p95 < 500 ms, ITL p95 < 50 ms
Cold start, no prep	5–15 min; with pre-pulled weights, 1–3 min

Quick mental math: 70B FP16 on one H100's bandwidth → 140 GB ÷ 3.3 TB/s ≈ 42 ms per decode step at batch 1; TP-8 divides it to ~5 ms plus all-reduce; FP8 halves it again.